AN2DL - Second Homework Report
AFJ

Jasmin Spinetto, Luis Felipe Epia Realpe

jasminspinetto, luisfelipeepiarealpe

277431, 258683

March 26, 2026

1 Introduction

This project focuses on the **semantic segmentation** of different elements belonging to **Mars terrain** gray-scale images using a **deep learning network**. The aim is to correctly identify and classify objects belonging to **four unbalanced classes**: Soil, Bedrock, Sand and Big Rock. There is a fifth class, Background, which is labeled but is not considered in the network model evaluation.

2 Problem Analysis

2.1 Dataset characteristics

The project involves two datasets, **D1** and **D2**. D1 was used for training and validation, it contains two matrices of 2615 elements of size 64 x 128. One contains the **images** and the other one the related pixel **labels** in integer labels format. D2 instead, was only used for testing, and therefore contains only one matrix of 10022 images without labels.
After compiling the model and predicting the labels on D2, the results were uploaded on a competition on *Kaggle.com*, where our model was applied to an unknown test set and marked based on the **Mean Intersection Over Union** metric.

2.2 Main challenges

- **Data Cleaning**: efficiently removing extraneous images.
- **Data Augmentation**: properly applying augmentation to both images and labels to get a richer D1 dataset.
- **Model Optimization**: starting from a simple U-Net model, creating a more elaborate network to increase performance.
- **Memory Efficiency**: managing the limited GPU internal memory.
- **Loss Function**: choosing the optimal loss function.
- **Low Quality Labels**: managing the low quality of some labels in the training set.

3 Method

3.1 Data Cleaning

In D1, there were around a hundred **intruder images** containing an overlapped alien head template. We found they all had the same exact label template, being therefore useless for our task. We used these specific labels as a template for comparison to automatically erase all the alien images.

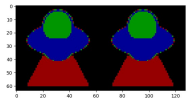

Figure 1: Alien Label Template

Other extraneous factors in images such as spaceship parts, astronauts parts or shadows and crawler traces were all labeled as background already, so we didn't take any action related to them.

3.2 Data Augmentation

In the case of semantic segmentation, geometric augmentation is somewhat trickier to apply, since operations as Rotate, Zoom and Flip, which move the image pixels, also need to be applied on the labels. We experimented with **Rotation** augmentation, both with the Scipy and OpenCV modules, but couldn't find an acceptable solution, since the rotation of images and labels seemed to randomly go in the same or opposite direction. **Zoom** augmentation wasn't relevant because objects in the dataset where already present in various sizes. We therefore only implemented **Flip** augmentation.

3.3 Memory Management

Since in segmentation networks the number of parameters increases very quickly with complexity, memory was a main bottleneck of our work. Even if the Out-of-Memory issues encountered where managed by TensorFlow, they could still decrease performance of the network. We tried using **Mixed Precision Policy** but it degraded the quality of our output. We also tried implementing Gradient Checkpointing but without success.

3.4 Loss Function Choice

We used **Dice Loss** in combination with Cross Entropy, the first to focus on the global overlap between predicted and true masks, the second to focus on pixel-wise classification accuracy. Since it focuses on global overlapping, Dice Loss is also useful to compensate the class imbalance of D1. It is defined as:

$$\text{Dice Loss} = 1 - \text{Dice Coefficient} \tag{1}$$

$$\text{Dice Coefficient} = \frac{2 * \text{Intersection} + \epsilon}{\text{Union} + \epsilon} \tag{2}$$

Where *Intersection* is the overlap between predicted and true labels, *Union* is the total count of predicted and true labels combined and ϵ is a small number added to avoid having an invalid division by zero. We also experimented with Generalized Dice Loss, which considers also class weights, but it didn't lead to better results.

3.5 Poor Quality Labels

To compensate the poor quality of the training set labels, which seemed to be too sharp and sometimes actually mismatched, we tried using **Label Smoothing** integrated in our Loss Function. However, this actually decreased performance. We therefore deduced that labels in the competition test set were of the same poor quality and that the competition test metric was sensitive to precise class assignment.

3.6 Metrics

To visualize better our model training results, we used **Visualization Callback** to print one chosen input image with its related true labels and predicted labels. This allowed us to supervise the model training progress. We also used the **Mean Intersection Over Union** metric and printed a graph tracking its progress over epochs on both the training and the validation set to understand the model's convergence pattern.

4 Experiments

In chronological order, the incremental main models we tried were:

- **Simple U-Net**: we began experimenting with the simplest U-Net model composed by a Downsampling of two U-Net blocks and an Upsampling path. A U-Net block is formed by a Convolution layer followed by a Batch Normalization and an Activation layer.

- **Data Augmentation**: we added augmentation during training. In particular, we implemented random flipping, contrast and brightness. A relevant factor was also the shuffling of the images inside the batches when creating the dataset, which avoided the network from learning the order of the images.

- **Deeper-ROP**: the first big improvement we got was by adding two additional layers to the U-Net and by adding a **ReduceOnPlateau** learning rate scheduler, which automatically decreases the learning rate when validation accuracy stagnates. It adapts the learning rate to the training phase: in earlier epochs the learning rate should be higher to enable large updates of weights, while it should be smaller in the later epochs to perform fine-tuning.

- **Dice Loss**: we used a weighted sum of Dice Loss and Sparse Categorical Cross Entropy as total loss function.

- **U-Net + ASPP**: another small improvement we obtained was by adding **Atrous Spatial Pyramid Pooling** as a bottleneck between encoding and decoding. It is composed by: *Atrous Convolutions*, *Global Average Pooling* and *Feature Concatenation*

- **U-Net Deeper**: U-net with 4 down/up blocks and a residual network after each u-net block of stack size of 3 with a small droput of 0.10 on the bottleneck layer.

- **U-Net Deeper no background**: Background predictions do not contribute to the scoring process, so we excluded them by slicing the predictions.

5 Results

Table 1: Best results obtained by fine-tuning various models:

Model	MeanIoU(Kaggle test)
1.Simple U-Net	0.13234
2.Data Augmentation	0.08134
3.Deeper-ROP	0.44529
4.Dice Loss	0.45629
5.U-Net + ASPP	0.47247
6.U-Net Deeper	0.51345
7.U-Net Deeper no background	0.66132

Predicted labels on a random chosen image from D2 with the various models reported in the table:

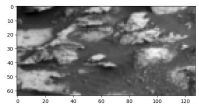

Figure 2: Test Image

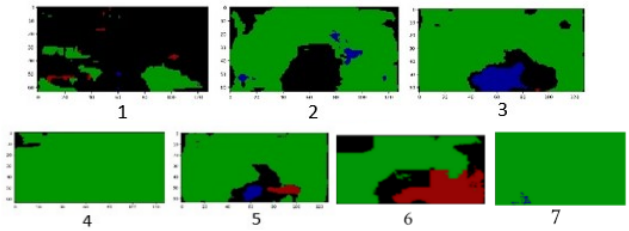

Figure 3: Test Image Labels

6 Discussion

Only adding augmentation actually reduces the performance of a simple model because it does not have enough capacity to learn the complex representation of data. We therefore managed to obtain better results by deepening U-Net.
Adding Dice Loss helped addressing the class imbalance issue.
By adding the Atrous Spatial Pyramid Pooling we managed to address the fact that single class objects where present with different sizes.
Overriding the background predictions led to a significant boost in performance while maintaining the exact architecture and hyperparameters.

7 Conclusions

In general, we created a stable and well-converging model that doesn't over-fit, however, even if the predictions on an unknown test set are not too imprecise, reaching **66%** MeanIoU, our network could still be improved. On this report, we only reference the model changes that led to improvements, however, we tried many different solutions without success. In particular, we weren't able to find a way to manage the bad quality of the given labels, which we feel is our biggest model weakness, and the class imbalance. We think that by making an even more thorough research, one could find better architecture templates for this specific project.

References

1. https://docs.monai.io/en/stable/losses.html
2. https://keras.io/api/callbacks/reduce_lr_on_plateau/
3. https://www.geeksforgeeks.org/u-net-architecture-explained/
4. https://www.researchgate.net/figure/The-overview-of-ASPP-module-It-consists-of-a-convolution-layer-with-a-kernel-size-of-1_fig5_376370640
5. https://medium.com/merantix-momentum-insights/label-efficient-semantic-segmentation-f6f601ff0a85
6. Ruirui Li, Wenjie Liu, Lei Yang, Shihao Sun, Wei Hu*, Fan Zhang, Senior Member, IEEE, Wei Li, Senior Member, IEEE. DeepUNet: A Deep Fully Convolutional Network for Pixel-level Sea-Land Segmentation, September 2017.

Group Members Contributions

1. Luis Felipe Epia Realpe:
   - Network development: tried different network models getting the first big improvements and achieved our best result.
   - Data cleaning: analyzed dataset images and labels and found the best ways to get rid of alien images.
   - Report error-checking and contribution by integrating some of the architectural techniques he tried.
2. Jasmin Spinetto:
   - Network development: also tried many different model architectures.
   - Parameter fine-tuning: fine-tuned the parameters on the best achieved model trying to get better results.
   - Report development: wrote the report by using Latex on Overleaf.